

Bachelor's Thesis
(Course Code: XB_40001)

Bypassing Google Play Protect

by

Mikołaj Sobczyk
(Student Number: 2775009)

Submitted in partial fulfillment of the requirements
for the degree of
Bachelor of Science
in
Computer Science
at the
Vrije Universiteit Amsterdam

July 18, 2025

Certified by .....
Herbert Bos
Prof. Dr. Ir.
First Supervisor

Certified by .....
Dyon Goos
PhD Candidate
Daily Supervisor

Certified by .....
Dyon Goos
PhD Candidate
Second Reader

Bypassing Google Play Protect

Mikołaj Sobczyk
Vrije Universiteit Amsterdam
Amsterdam, NL
m.sobczyk@student.vu.nl

ABSTRACT

Google Play Protect is the primary automated security system protecting billions of Android users from malware on the Google Play Store. Regardless of the fact that it has been over a decade since the last study that examined its predecessor, documented in "Adventures in the Bouncerland" [16], where researchers were able to uncover dangerous security flaws, the current methodology of Google's new scanning systems is unexplored. This study presents a detailed analysis of Google Play Protect's (GPP)[11] dynamic analysis capabilities, by creating and deploying an app, which sends the logs to the server to collect and analyze the data about the device that is running it. We have evaluated current GPP's capabilities, including interaction patterns, interaction depth, as well as environmental characteristics, to answer whether the security level of Android devices has improved since 2012. The findings reveal that some of the vulnerabilities found in the previously mentioned research are still present in the current app vetting system. Dynamic analysis uses predictable environmental characteristics, making it simple to detect if the app is running on an actual device. It also uses unsophisticated interaction patterns, which open the door for logic bombs. Analysis of 33 navigation events demonstrates the limitations of current anti-malware solutions. Established evasion techniques used by Percoco and Schulte [16] could be still viable. The use of simple environmental detection combined with dynamic payload delivery could probably evade GPP's detection. This research provides the first evaluation of Google Play Protect since its deployment, setting baselines for future research in mobile security.

1 INTRODUCTION

The rapid increase in mobile malware is one of the most significant threats to the current cybersecurity world [26]. It stems from the fact that mobile devices have become an essential part of our lives. Those devices simplify lives by giving easy access to knowledge, communication, finance, and more. However, this makes modern smartphones a gold mine of sensitive data, from private messages to banking and health details. As convenient as it is for users, it makes them a target for malicious actors, who are trying to steal their data. Google Play Protect[11] stands as the first line of defense for millions of Android users worldwide. It is a built-in security feature on Android devices that scans applications before and after downloading from the Google Play Store, looking for Potentially Harmful Applications. Understanding this system is crucial for mobile security researchers. The evolution of malware detection systems, particularly those using dynamic analysis, has been an ongoing arms race between security providers and cybercriminals. While static analysis can identify patterns in code that could be potentially harmful to the user, dynamic analysis allows for detecting threats that have been hidden by employing obfuscation, encryption, and runtime code loading. However, dynamic analysis is not exhaustive. There are multiple techniques that were discovered to get past it, such as logic or time bombs. Those methods are based on a delayed activation mechanism to make sure that the malicious activities are run on actual devices, not a simulated and observed environment. Due to that fact, its effectiveness depends heavily on the ability to trigger malicious behavior during the limited analysis window. Despite Android's dominance in the mobile device market, with over 3.3 billion active devices and a market share exceeding 71.85% worldwide, there is a significant research gap in understanding how Google Play Protect's dynamic analysis system works [14]. The mobile market has evolved dramatically since the last study on Google's protection against malware, which was performed over a decade ago, with research about Google Bouncer in 2012 [16]. This research gap leaves Android users, developers, and security researchers without much or any knowledge of how the app vetting procedure is performed. Due to Android's open-source environment, it is an attractive target for malicious actors. The lack of transparency in GPP's mechanism leaves security researchers without the invaluable insights needed to identify malware infiltration risks and recognize malware detection's shortcomings. The absence of recent studies looking into GPP's behavior introduces a blind spot that can be exploited by attackers. This is a valuable insight, taking into consideration the importance of security for Android users, who put their trust in the official app marketplace. By means of an empirical study, the first in over a decade, we address this gap, closing the research gap on understanding GPP. In the technological industry, 10 years is enough time to turn around the whole system. With this investigation, we make several key contributions to the mobile security research industry:

- Assessment of current capabilities: Through technical experimentation with a specifically developed Android application, this research provides data on GPP's dynamic analysis interaction with the app in the process of being published to billions of users.
- Evolution analysis: By comparing the gathered results to the previous study in this domain from 2012, we check if the system has changed over that time.
- Evasion technique evaluation: We dive into examining the effectiveness of logic and time bombs against the protection system, providing insights for both defensive and offensive security research.

We plan to open this project so that other security researchers can build upon it, using the resources from this study.

2 BACKGROUND

Static and Dynamic Analysis. Malware detection systems use 2 primary techniques for identifying malicious activities in scanned applications, both have their advantages and shortcomings.

Static Analysis. This method examines the source code and the binary resources used, as well as metadata. All of that happens without executing the code. It is computationally efficient when dealing with large codebases, but is limited when faced with detecting polymorphic code, runtime code generation, obfuscation techniques, and time/logic bombs.

Dynamic Analysis. This approach simply executes the software and monitors its behavior, looking for suspicious activities, unexplained network traffic, or unexpected file changes. It can be more resource-intensive but can detect more complex behavior, such as code injection, memory analysis, runtime code generation, and obfuscation. However, with a limited time window, it may lack the ability to interact with the application deeply enough to trigger hidden behavior. The malicious code can also stay inactive if an analysis environment is detected.

Modern malware detection systems combine both methods to improve the accuracy of detection.

Sandboxing and Virtual Environments. Dynamic analysis techniques rely on sandboxing, which allows for executing unknown software within a safe, isolated, and controlled environment. This simultaneously prevents damaging the system and allows for detailed monitoring of activities performed in the environment. Those sandboxes usually consist of virtualized or emulated Android devices that replicate real execution conditions.

Effective sandboxes should balance replicating real conditions to trick the malware into executing the malicious code while maintaining isolation. Security researchers could benefit from improving the stealthiness of sandboxes, as malicious actors have learned to differentiate between real and virtual environments and use them to evade detection by acting benign when they know they are being watched.

Logic Bombs. Logic bombs are effective, yet simple to implement evasion techniques, targeted at dynamic analysis systems. This technique relies simply on waiting to execute malicious activities until they confirm that they are being run by a real user on a real device. It is achieved by adding predetermined conditions, such as:

- Interaction Counters: Requires multiple genuine interactions with the app to execute the code.
- System dependencies: Activating only when a specific system component is present.

Logic bombs are dangerous due to their flexibility. Conditions can be sophisticated enough that the dynamic analysis may not be able to meet the circumstances specified by the malicious actors.

Time Bombs. This technique relies on simply waiting out the limited execution window of the dynamic analysis. It can execute the hidden code hours after the initial run of the application, or use calendar-based activation, allowing it to execute on the specified day of the month.

Environment Detection. Malware creators use environment detection to assess if the application is running in a sandbox or a real environment. This allows them to differentiate between real and virtual environments and use them to evade detection. Common strategies include:

- System Property Analysis: By checking device properties, build information, hardware specifications, runtime, and other characteristics that may indicate the presence of a sandbox. [15]
- Network Environment Analysis: By monitoring network traffic and looking for signs of the virtual environment. Checking IP addresses that may point to Google's domain is a common practice. [20]

Additional Evasion Techniques Beyond The Scope Of This Study. We focused on environment detection, logic bombs, and time bombs. However, attackers have access to more evasion techniques, including:

- Encrypted or obfuscated code: Malicious code can be encrypted or obfuscated, making it hard to detect with static analysis.
- Dynamic code loading: Loading code from external sources post-installation allows malicious actors to deliver unwanted functionalities after the scanning procedure.
- Native code: With the use of the Android NDK, attackers can implement malware in native C/C++ code, which is harder to detect and observe during scanning.
- Permission escalation: Apps may first request minimal permissions in order to look benign. However, they can later be updated and request more permissions.

These strategies show that bypassing static and dynamic analysis is possible with an increasing number of techniques.

3 THREAT MODEL

Attacker Goals. We assume an attacker with the primary goal of distributing malware through the Google Play Store, in order to achieve it, the attacker has to get past Google Play Protect. With Google Play Store as the main distribution channel, the malicious actor has to focus on Google's official vetting procedures in order to release the malicious application. The attacker designs the app in a way that behaves benignly during the scanning procedure. Post successfully completing the vetting process, the application finally starts executing its malicious behavior, after ensuring it is placed on an actual device. To achieve this result, the attacker leverages evasion techniques, such as environment detection, time bombs, and logic bombs.

Victim Profile. The intended victim is an everyday Android user who downloads applications from the Google Play Store, trusting its security. The user has not enabled app installations from unknown sources and trusts that apps approved by GPP are safe.

Security Assumptions. We assume that:

- Google Play Protect uses automated dynamic analysis in a sandboxed environment during the upload process.
- The test environment has predictable characteristics, such as IP address and uptime. It also has a limited analysis window.

- After the app successfully passes the review, it is assumed to work without further investigations.

Threat Scenario. A malicious app is submitted to the Google Play Store and passes the vetting procedure by acting benign while being observed. Only after being installed on an actual device and fulfilling all preset conditions for delayed activation, it starts acting maliciously, exploiting the trust Android users put in the official app marketplace.

4 DESIGN

Developer Account Setup and Publishing Procedure. In order to get the application to be scanned by Google Play Protect, we had to complete the formal procedures by following Google's standard registration procedure. The procedure included verifying the user, with the requirement of providing documents and proving identity and address. The next step was paying a 25$ registration fee, ending with accepting the Google Play Developer Distribution Agreement [12].

For application preparation, we created a PennyTracker, a simple expense tracker app. The formal publishing procedures required to prepare the app description, screenshots, category selection, privacy policy, and terms of service, as well as proper age rating and content declarations. The publishing process targets the closed testing phase, which Google introduced for accounts created after 2023. This imposes a requirement of going through the closed testing phase, which relies on 20 different Android users testing the app for 14 consecutive days. It is a mandatory step to eventually publish the app to the Google Play Store. We target the initial phase of this process, where the signed APK is uploaded, through the Google Play Console. Importantly, Google Play Protect analysis happens immediately after submitting the application for that step, allowing the built data collection system to capture its dynamic analysis interactions with the app, as well as device information. The scope of this research stops at the closed testing submission, as it has already received all the data from scanning inside the Google Play Protect system. The extended 14-day testing period becomes irrelevant for research purposes, as the scanning is already completed.

Data Collection Strategy. The tracking infrastructure uses UserTrackingService, a singleton service that captures interaction data. This service records screen navigation with precise timestamps, user interface interactions, including button clicks, as well as form submissions, and application lifecycle, including start, stop, and quit states. Additionally, we have implemented a DeviceInformationCollector that gathers environment data. The data collection focuses on recording everything that may be relevant to capturing malicious applications. It is meant for manual analysis after successfully gathering all the needed information. The data structure uses a simple message-based format with a user ID for keeping track of the user's actions. Each event is associated with a user ID, and the system captures both the device information as JSON strings and simple timestamped event messages. Device information includes hardware features, build class, network data, environment, locale, and runtime.

Server Setup and Data Transmission. The backend infrastructure is hosted on the Render [19] platform at https://server-oiej.onrender.com, setting up a RESTful API endpoint for collecting the data through the /report route. The communication is implemented with HTTP transmission, with JSON payload processing. The data transmission protocol is implemented using a Retrofit-based service to ensure data upload. The data processing pipeline focuses on post-collection manual evaluation of the collected data. This includes manual classification of behavioral patterns to find automated interaction characteristics. Environment data is crucial to find out if there is a straightforward way to distinguish whether the application is running on a real device or a simulated environment. The manual analysis approach allows for a deep interpretation of the collected data and identification of patterns that might indicate automated testing, which allows for checking the sophistication of Google Play Protect's dynamic analysis system.

Design Motivation and Alternatives. The main design decisions of this project were to create a custom Android application and publish it through the official Google Play Console. These choices resulted in achieving full control over the app's behavior and ensuring compatibility with Google Play Protect's scanning procedure. Alternative approaches, such as analyzing existing malware found on the Google Play Store and reverse engineering it, were considered. However, those options would not provide insight into the scanning procedure. We chose Jetpack Compose, as it is a recommended UI toolkit, which increases legitimacy during static analysis. We also believed that recommended technologies would be most compatible with Google's defense system, allowing the analysis to perform with the highest accuracy. For the backend, FastAPI was chosen due to its simplicity, as, for the purposes of this research, no sophisticated functionality was needed for the backend system.

5 IMPLEMENTATION

Android Application Development. We have chosen Jetpack Compose as the primary UI framework for strategic reasons. It is Google's recommended UI toolkit[3], it represents current Android development best practices[5], making the app appear legitimate to the static analysis techniques. The framework's built-in Material 3 integration ensures compliance with Google's design guidelines[5], also simplifying the development experience. Room database was chosen for local data persistence as Google's officially recommended database tool [4]. These choices minimize potential static analysis flags. The application uses a Model-View-ViewModel architecture pattern to separate UI and data layers. This is an industry standard. We have employed the Repository pattern for centralized data access. Jetpack Navigation handles screen management, essential for tracking users' navigation inside the app. The user interface implements a subtly complex multi-layer architecture to test interaction depth. The main navigation layer includes a navigation bar for primary screen access and a floating action button for expense adding. The secondary interaction layer includes the Add Expense screen, accessible via FAB click, containing form elements, such as Name, Amount, Date, Category, and Expense/Income. This is implemented using simple input fields for Name and Amount, as well as a drop-down for Category, a date picker dialog, and an Expense/Income toggle chip. After filling in all of that, there is a Save button. This structure provides valuable insight into the dynamic analysis sophistication in reproducing legitimate user behavior.

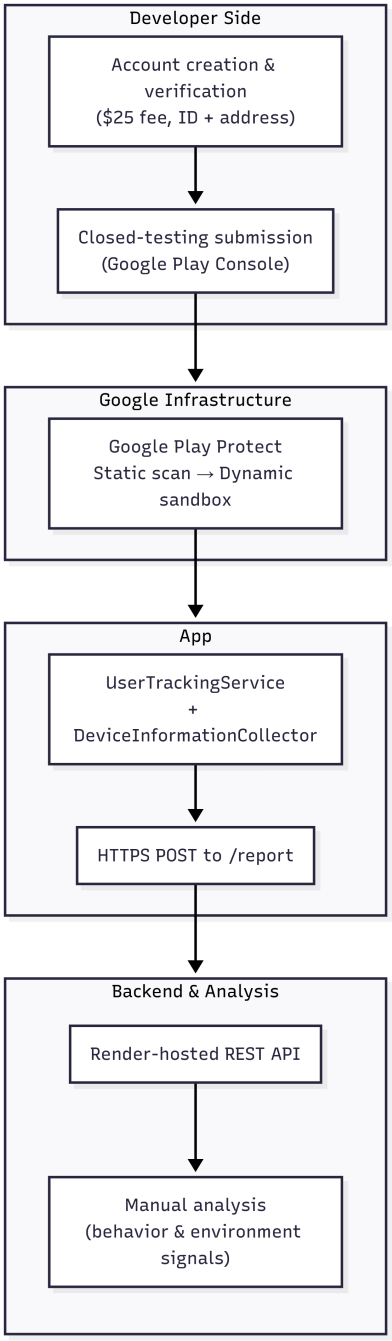


Figure 1: High-Level Overview Flowchart

The tracking implementation is built using a singleton UserTrackingService for event capturing. This service detects screen changes, using Compose Navigation listeners, monitors the application's life cycle events through Android's lifecycle components, and logs everything throughout the execution. The service captures all activities with precise timestamps. This is the structure and all the information collected from the device:

- Build Class: Manufacturer, Model, Brand, Device, Fingerprint, Android Version, SDK Version, Hardware, Board, Host, Bootloader, Radio Version, Supported Abis
- Locale: Language, Country, Timezone
- Network: Sim Operator, Network Operator, Local IP, Public IP
- Environment: Emulator, ADB Enabled
- Runtime: Uptime

Event tracking captures the following actions:

- App Started
- App Paused
- App Quit
- Screen Navigation: Add Expense
- Screen Navigation: Analytics
- Screen Navigation: Home

Except for that, each of those logs contains an exact timestamp of the event.

Server Infrastructure Implementation. The backend infrastructure uses Python [17] with the FastAPI framework [9], hosted on the Render platform [19]. We have chosen FastAPI because of its simplicity, which was enough for this use case. The Render platform provides managed hosting. The API implements a simple RESTful architecture with a \report endpoint for uploading reports, and a \reports endpoint for retrieving the collected information. This design focuses on simplicity and provides all the necessary functionality for this purpose. The server processes incoming data synchronously, providing instant confirmation of stream transfer. Post-collection analysis is based on manual interpretation of timing and interaction patterns. The environment data is also analyzed manually to get insights into its sophistication.

Build and Deployment Configuration. The application build process considers Google Play's requirements for app submission. It was configured to generate an Android App Bundle (.aab). This was required by Google Play Console [2] for submitting purposes. The build process includes code signing using a release key. The final .aab file was generated through Android Studio's [1] Generate Signed Bundle process, ensuring all necessary certificates and configurations and enabling Google Play Protect to analyze the app correctly.

6 EVALUATION

6.1 Collected Data Analysis

The system tries to appear as a legitimate device rather than an emulator, however, there are a few shortcomings that allow malicious actors to detect that it is running in a simulated environment. The IP **66.102.7.4** links back to Google, the same as discovered 13 years ago in "Adventures in the Bouncerland" [16]. The runtime is only 5 minutes and 40 seconds, so it is fair to assume that it is not a real device. Those two simple characteristics can be used by hackers to detect whether the app is running on an actual device, showing a significant hole in the security system.

Interaction Pattern Analysis. The dynamic analysis log reveals a highly systematic, but superficial interaction pattern, which can be found in the sequence graph in 2. The application is scanned for

Table 1: Detailed Device, Locale, Network, and Runtime Environment Information of GPP's emulated device

Category	Key	Value
Device	manufacturer	OnePlus
	model	OnePlus8Pro
	brand	OnePlus
	device	OnePlus8Pro
	fingerprint	OnePlus/IN2023/OnePlus8Pro:11/QKR1.191246.002/2006701342:user/release-keys
	android_version	11
	sdk_int	30
Hardware	hardware	sunfish
	board	cutf
	host	ubuntu-10
	bootloader	unknown
	radio_version	1.0.0.0
	supported_abis	x86_64, arm64-v8a, x86, armeabi-v7a, armeabi
Locale	language	en
	country	AU
	timezone	Australia/Sydney
Network	sim_operator	Vodafone
	network_operator	Vodafone
	local_ip	192.168.97.2
	public_ip	66.102.7.4
Environment	emulator	false
	adb_enabled	false
Runtime	uptime_ms	340192
	uptime_readable	05:40

6 minutes and 15 seconds, which would allow cybercriminals to set a simple time bomb, evading detection by delaying malicious behavior. There were multiple applications that started and stopped during the duration of the scan, which is a proper behavior, as that could be a trigger to execute unwanted code. However, there were no form submissions or data entries, showing a lack of sophistication in testing methodology.

Exploitable Patterns. The dynamic analysis only interacted with navigation elements and never submitted actual data through the form. This is a crucial finding, as it shows that the dynamic analysis system is only able to interact with the first layer of the application, which can be easily exploited by following strategies:

- Malicious functionality triggered by form submissions would remain undetected
- Logic bombs activated by specific input combinations
- Server-side malicious payloads delivered only after legitimate data submission

Predictable Navigation Sequence. The dynamic analysis system is highly repetitive in its interactions, and it focuses only on main-screen navigation. This could allow malicious actors to exploit hidden functionality in sub-menus or secondary flows.



Figure 2: Sequence Graph of GPP's dynamic analysis interaction with the PennyTracker

No Deep Feature Testing. Several important interactions were missing, including:

- Form field input, such as description and amount
- Category selection drop-down
- Date picker dialogue window
- Expense/Income toggle
- Submit button
- Edit/Delete operations via overflow menu

This lack of depth in testing allows simple logic bombs that could be triggered by waiting for specific form inputs, which would look like a legitimate user.

Limited Session Depth. Short analysis sessions with frequent app restarts create a potential time bomb evasion, it could be set to run the malicious code after a specific number of application launches or after several days.

6.2 Comparison with "Adventures in the Bouncerland"

It has been 13 years since the previous research examining Google's defense from malware. During that time, the field of cybersecurity has evolved considerably. This section shows the differences made to Google's dynamic analysis system over the last 13 years. We have identified some patterns that continue to be vulnerable.

Table 2: Comparison Between 2012 and 2025

Aspect	Bouncer (2012)	Google Play Protect (2025)
Detection Timing	Immediate upon upload	Still immediate upon upload
Analysis Duration	<30 seconds per scan	6 minutes and 15 seconds
Behavioral Detection	Minimal depth	Still limited interaction depth
Evasion Success	Successfully bypassed	Similar vulnerabilities persist

Some vulnerabilities haven't been patched since the last research in that area. The following vulnerabilities can be exploited easily:

- Short Analysis Window: The application is scanned for only a few minutes, and it is scanned only once.
- Surface-Level Testing: No deep interactions were triggered, only the navigation buttons and Floating Button on the Main Screen.
- Predictable Environment: Static device profiles and network configurations.
- Evasion Techniques: This system can be evaded in the same manner as in the previous research, by a simple IP-based detection, as it still links to Google services.

On the other hand, the Android versions were updated, as now it uses Android 11, along with SDK version 11, which is a version from 2020. There is still some room for improvement, but it is a good sign that there has been some progress in that area. An interesting thing is the geolocation data, as it shows that the scan was run from Australia, which introduced some variability.

Recommendation for Improvement. There are multiple changes that could be implemented for higher quality and efficiency in malware detection. Google Play Protect would benefit from:

- Extended Analysis Duration: This solution could improve efficiency in detecting malware using delayed activation.
- Randomize Interaction Patterns: Implement non-predictable user simulations.
- Deep Feature Testing: Extend the interactions to deeper features than the current navigation through main screens.
- IP Address: Solving the issue of the Public IP Address pointing back to Google.

Implications. This research showcases that mobile application security remains imperfect, with a lot of room for malicious actors to exploit. The presence of vulnerabilities found in "Adventures in the Bouncerland" is a shocking discovery, as it shows that techniques have been well documented and publicly available to everyone, even to the people who want to exploit them. This is a serious threat that should have been addressed a long time ago. The findings show that simple environmental detection, combined with logic bombs, has a high chance of being undetected by the current system, highlighting the need for more sophisticated detection mechanisms. The shallow interaction patterns represent a failure in addressing the well-known and researched evasion methods that are present in many malicious applications that were placed on the Google Play Store.

Data Collection Limitations. The implemented tracking solution managed to collect the device's metadata, interaction logs, and navigation patterns. The focus was placed on visible behavior, on application-level events. No root access was used on the Play Protect environment, thus restricting the ability to retrieve low-level data such as:

- System call traces
- File system or memory usage
- Background services
- Runtime permission prompts

Despite these limitations, the collected data was enough to detect patterns that can be interpreted as superficial, which supports the conclusion about the system's vulnerabilities.

7 DISCUSSION

7.1 Limited Application Coverage

This study is focused only on a single expense-tracking application, thus it may not be able to represent how Google Play Protect handles different categories of mobile applications. Applications from different categories may trigger different behavior in the dynamic analysis. Social media, games, or productivity tools might be treated entirely in a different way, based on their functionality, permissions, or user base. Future research should look into dynamic analysis across different categories to establish whether the patterns are the same for all applications.

7.2 Closed Testing Barrier

The Google Play Console's Closed Testing requirement, which mandates 20 different users test an application for 14 consecutive

days before public release, created a significant limitation for this study. This phase remains unexplored in our methodology, leaving questions about whether additional or more sophisticated tests occur at a later stage of the publication procedure. Future work should recruit 20 legitimate testers to document any additional scanning that may occur in later phases.

7.3 Time Constraints

This research was conducted over a limited time window, it might not have captured long-term changes in Google Play Protect's behavior or updates to its security system. Those systems usually are continuously evolving and changing. Studies over a span of years would be able to capture the ongoing progress of mobile security development.

7.4 On-Device Protection

The subject of our study, which is Google Play Protect in the cloud, is not the last defense against malicious applications. There is another mechanism ensuring the safety of Android mobile devices. That mechanism is the on-device protection, which includes a daily scanning procedure that aims to find Potentially Harmful Applications, no matter their source. Resulting in additional layer of security, and a way to catch malicious applications that managed to go under the radar through the first scanning procedure. This is another area with lack of documentation that should be looked into with empirical research.

7.5 Interaction Depth Limitations

The application created for this study's purpose was a simple expense tracking app, with limited functionality and sophistication. More advanced applications with complex workflows, permissions usage, or advanced UI components might reveal new insights into Google Play Protect's capabilities. This should be investigated in future research.

8 RELATED WORK

Research Documenting Malware in Google Play Store. Many Android malware have made it to the Google Play Store. Zhou and Jiang [24] conducted one of the earliest studies, where they analyzed 1260 malware samples from different Android markets between 2010 and 2012. Their work showcased that malicious actors often use repackaging to evade detection.

Work shown in "Dendroid: A text mining approach to analyzing and classifying code structures in Android malware families" [22] documented the presence of more sophisticated malware families in the Google Play Store.

Previous Study Analyzing Google Play Protect's Behavior. This topic is not researched enough. The last study that analyzed how the Google Bouncer, which is the predecessor of Google Play Protect, was conducted in 2012 in "Adventures in the Bouncerland"[16]. This study provided fundamental insight into the app vetting procedure. Percoco and Schulte documented Google Bouncer's characteristics by deploying malware that they created. Their SMS Bloxor application successfully evaded detection while adding more malicious features. The study revealed that the environment was an emulated HTC Sapphire with predictable IP address, which linked back to Google, with a very limited analysis duration of less than 30 seconds per scan. This study established many key insights into this field of research, some of which are still relevant 13 years later. They found that the analysis environment was only able to scan it on upload, with a lack of any post-installation monitoring, making delayed activation effective. The methodology used incremental malicious functionality addition across many versions that were uploaded to the Google Play Store. The researchers successfully implemented IP-based evasion, ensuring that the malicious behavior would not be triggered while being observed. Since the 2012 study, no one has touched upon that field, which leaves a massive research gap. This is dangerous, due to the relevance of Android in the mobile device market, with billions of users.

Known Evasion Techniques. Mobile malware evasion techniques have gone a long way since the beginning of the Android application markets, especially in approaches targeting automated analysis systems. The following paragraphs will focus on logic bombs, time-based activation mechanisms, and environmental detection, as those techniques are most relevant to this study's scope.

Logic Bombs. Logic bombs represent one of the most effective evasion techniques while being simple to implement. These mechanisms delay malicious activity until specific conditions are met, those conditions are set there to ensure that the application is run on an actual device. "A survey of mobile malware in the wild" [10] investigated early implementations of logic bombs in Android malware, providing insight into how applications acted benign until triggered by user interactions, system events, or other conditions.

Zhou *et al.* [25] analyzed how logic bombs are implemented in different malware families, trying to identify commonly used trigger mechanisms, such as SMS receipts, incoming calls, and location changes. Their study revealed that logic bombs successfully evaded automated analysis systems that were based on short observation.

Eck and others in their research [8] developed dynamic taint analysis for Android applications, with which they were able to reveal sophisticated evasion techniques, such as conditional data leakage based on user behavior patterns. This work provided insight into Android malware's sophisticated approach to evasion techniques, based on complex decision trees that activated malicious functionality after confirming the presence of a real user.

Time-Based Activation. Techniques based on time delay were found effective against scanning systems, using short scans. Research by Rastogi and others [18] documented malware families that used time-based triggers, such as absolute time delays, usage-based activation counters, and calendar triggers.

The DroidDream family, analyzed by Schmidt *et al.* [21], used sophisticated timing mechanisms, which activated malicious code after a specified period of legitimate usage. These applications would remain benign for days or weeks before starting malicious activities, which allowed them to bypass any automated analysis.

Grace and others in their study [13] looked into time-based evasion techniques across large datasets of malware, successfully identifying patterns in activation delays and their effectiveness against automated scans. This study revealed that even short delays (24-48 hours) were enough to evade most automated analysis

systems, while longer delays, measured in weeks to months, were almost completely efficient in evading detection.

Environmental Detection. Environmental awareness is another critical evasion technique relevant to automated analysis. Vidas, Timothy, and Christin [23] documented fingerprinting techniques that malware could use to detect analysis environments, including hardware characteristics, network configuration, and system properties.

Research performed by Bläsing *et al.* [6] examined malware, which first checked the environment, in order to activate the malicious activity only on real devices. The applications targeted by this study were checked for emulator characteristics, debugger presence, as well as specific system properties to verify that they were ready to act maliciously without the fear of being detected.

Burguera and others [7] analyzed environment-dependent malware behavior, showing that applications adjusted their functionality based on system characteristics. Their research displayed that sophisticated malware could combine hardware fingerprinting with behavioral analysis to verify the real environment.

9 CONCLUSION

This research provides empirical evidence that Google Play Protect's dynamic analysis capabilities are still limited, despite over a decade since the shortcomings of Google Bouncer were publicly examined, some of those flaws are still there. Through the controlled deployment of the PennyTracker application and systematic analysis of Google Play Protect's behavioral patterns, we found that the defense mechanisms against Android malware are still flawed. The investigation revealed different critical vulnerabilities in GPP's analysis. The dynamic analysis environment uses predictable characteristics, including shallow interaction patterns, short analysis windows, and predictable IP addresses. The most important flaw is the lack of interaction with form submissions, drop-down windows, and input fields. This allows malicious actors to implement delayed activation mechanisms and deep interaction triggers for executing unwanted behavior. The comparison with the 2012 findings shows consistency in fundamental weaknesses. While the technical infrastructure for uploading the application to the market has improved since then, by introducing many steps, the scanning procedure still lacks sophistication, leaving the door open to many threats for Android users. It is essential to realise that detecting malicious behavior is difficult, even impossible. Malware authors and their techniques get better over time. Since there is no signature or behavior that defines all malware, evasion is still possible. The problem lies in assessing what behavior is, in fact, malicious. Therefore, detection systems should be a part of the strategy, not the only line of defense.

REFERENCES

[1] Android Developers. n.d.. Android Studio. https://developer.android.com/studio. Accessed: 2025-06-28.

[2] Android Developers. n.d.. Google Play Console. https://developer.android.com/distribute/console. Accessed: 2025-06-28.

[3] Android Developers. n.d.. Jetpack Compose – Android Developers. https://developer.android.com/compose. Last accessed: 2025-06-29.

[4] Android Developers. n.d.. Room – Android Training: Data Storage. https://developer.android.com/training/data-storage/room. Last updated: 2025-06-26; accessed: 2025-06-29.

[5] Android Developers. n.d.. UI Development – Android Developers. https://developer.android.com/develop/ui. Last accessed: 2025-06-29.

[6] Thomas Bläsing, Leonid Batyuk, Aubrey-Derrick Schmidt, Seyit Ahmet Camtepe, and Sahin Albayrak. 2010. An Android Application Sandbox system for suspicious software detection. In *2010 5th International Conference on Malicious and Unwanted Software*. 55–62. https://doi.org/10.1109/MALWARE.2010.5665792

[7] Iker Burguera, Urko Zurutuza, and Simin Nadjm-Tehrani. 2011. Crowdroid: behavior-based malware detection system for Android. In *Proceedings of the 1st ACM Workshop on Security and Privacy in Smartphones and Mobile Devices* (Chicago, Illinois, USA) *(SPSM '11)*. Association for Computing Machinery, New York, NY, USA, 15–26. https://doi.org/10.1145/2046614.2046619

[8] William Enck, Peter Gilbert, Seungyeop Han, Vasant Tendulkar, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. 2014. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. *ACM Trans. Comput. Syst.* 32, 2, Article 5 (June 2014), 29 pages. https://doi.org/10.1145/2619091

[9] FastAPI Contributors. 2018. FastAPI: high-performance web framework for building APIs with Python. https://fastapi.tiangolo.com/. First released Dec5,2018; version0.115.14 released Jun26,2025; accessed 2025-06-29.

[10] Adrienne Porter Felt, Matthew Finifter, Erika Chin, Steve Hanna, and David Wagner. 2011. A survey of mobile malware in the wild. In *Proceedings of the 1st ACM Workshop on Security and Privacy in Smartphones and Mobile Devices* (Chicago, Illinois, USA) *(SPSM '11)*. Association for Computing Machinery, New York, NY, USA, 3–14. https://doi.org/10.1145/2046614.2046618

[11] Google Developers. n.d.. Google Play Protect. https://developers.google.com/android/play-protect. Accessed: 2025-06-28.

[12] Google LLC. 2024. Google Play Developer Distribution Agreement. https://play.google.com/developer-distribution-agreement.html. Effective February5,2024; Accessed: 2025-06-29.

[13] Michael Grace, Yajin Zhou, Qiang Zhang, Shihong Zou, and Xuxian Jiang. 2012. RiskRanker: scalable and accurate zero-day android malware detection. In *Proceedings of the 10th International Conference on Mobile Systems, Applications, and Services* (Low Wood Bay, Lake District, UK) *(MobiSys '12)*. Association for Computing Machinery, New York, NY, USA, 281–294. https://doi.org/10.1145/2307636.2307663

[14] Naveen Kumar. 2025. *Android Usage Statistics 2025 – Market Share & Versions.* https://www.demandsage.com/android-statistics Accessed: 2025-06-22.

[15] Tamas Lengyel. 2015. Malware Collection and Analysis via Hardware Virtualization. (12 2015).

[16] Nicholas J Percoco and Sean Schulte. 2012. Adventures in BouncerLand: Failures of Automated Malware Detection within Mobile Application Markets. Black Hat USA.

[17] Python Software Foundation. 2001–2025. Python Home Page. https://www.python.org/. Last updated: Jun 29, 2025; accessed: 2025-06-29.

[18] Vaibhav Rastogi, Yan Chen, and William Enck. 2013. AppsPlayground: automatic security analysis of smartphone applications. In *Proceedings of the Third ACM Conference on Data and Application Security and Privacy* (San Antonio, Texas, USA) *(CODASPY '13)*. Association for Computing Machinery, New York, NY, USA, 209–220. https://doi.org/10.1145/2435349.2435379

[19] Render. n.d.. Render: Cloud Application Platform. https://render.com/. Accessed: 2025-06-28.

[20] Anandaram S., Ashik Mathew, Jyothish A., Vinod P., and Francesco Mercaldo. 2021. Hide and Seek Game: A Machine Learning Approach for Detecting Malicious Samples in Analysis Environment. In *Proceedings of the 2021 Thirteenth International Conference on Contemporary Computing* (Noida, India) *(IC3-2021)*. Association for Computing Machinery, New York, NY, USA, 349–354. https://doi.org/10.1145/3474124.3474211

[21] A.-D. Schmidt, R. Bye, H.-G. Schmidt, J. Clausen, O. Kiraz, K. A. Yuksel, S. A. Camtepe, and S. Albayrak. 2009. Static Analysis of Executables for Collaborative Malware Detection on Android. In *2009 IEEE International Conference on Communications*. 1–5. https://doi.org/10.1109/ICC.2009.5199486

[22] Guillermo Suarez-Tangil, Juan E. Tapiador, Pedro Peris-Lopez, and Jorge Blasco. 2014. Dendroid: A text mining approach to analyzing and classifying code structures in Android malware families. *Expert Systems with Applications* 41, 4, Part 1 (2014), 1104–1117. https://doi.org/10.1016/j.eswa.2013.07.106

[23] Timothy Vidas and Nicolas Christin. 2014. Evading android runtime analysis via sandbox detection. In *Proceedings of the 9th ACM Symposium on Information, Computer and Communications Security* (Kyoto, Japan) *(ASIA CCS '14)*. Association for Computing Machinery, New York, NY, USA, 447–458. https://doi.org/10.1145/2590296.2590325

[24] Yajin Zhou and Xuxian Jiang. 2012. Dissecting Android Malware: Characterization and Evolution. In *2012 IEEE Symposium on Security and Privacy*. 95–109. https://doi.org/10.1109/SP.2012.16

[25] Yajin Zhou, Zhi Wang, Wu Zhou, and Xuxian Jiang. 2013. Detecting passive content leaks and pollution in android applications. In *Proceedings of the 20th network and distributed system security symposium*. 1–16.

[26] Qingyi Zhu, Xuhang Luo, Yuhang Liu, Chenquan Gan, Yu Wu, and Lu-Xing Yang. 2024. Impact of cybersecurity awareness on mobile malware propagation: A

dynamical model. *Computer Communications* 220 (2024), 1–11. https://doi.org/10.1016/j.comcom.2024.03.017